Summary of Testing Results from Complex_Analysis_Two_Allele_Time_Invariant_Fitness_Case as of Oct 20

Previous results:

Haldane's fitness, $1 - \Psi = e^{-(s+1)\Psi}$, is significant within population genetics. A closed form solution has not appeared widely in the biological literature. Xia (2025) expresses the closed form probability of extinction based on Haldane's Poisson distribution law using Lambert W. We use this closed form solution here for the first time within the study of biological noise to analyze probability of fixation (see Figure 2).

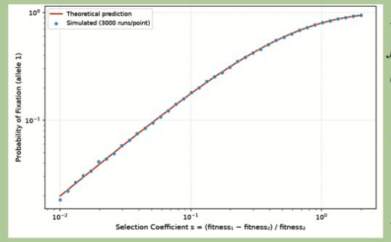


Figure 2: Probability of fixation as a function of selection coefficient, simulated compared to Haldane's fitness. Works for 1 beneficial allele starting in large pop. (5000 here). Our closed form expression

$$\Psi(s) = \frac{W_0\left(-(s+1)e^{-(s+1)}\right)}{s+1}$$

verifies the accuracy of our two-allele, time invariant fitness model.

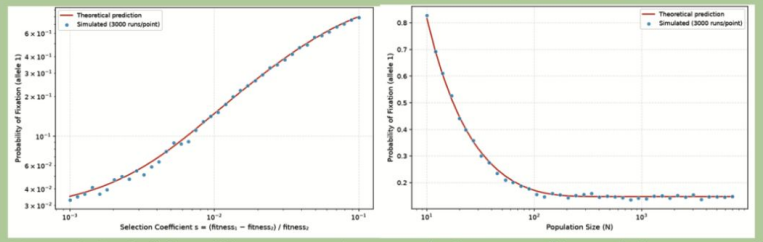


Figure 3: Probability of fixation as a function of selection coefficient (left, pop. size of 300) and population size (right, selection coefficient of 0.01), simulated from two-allele, time invariant fitness model compared to expected $P = \frac{1 - e^{-2s}}{1 - e^{-2Ns}}$. *Prediction for introduction of some weakly beneficial allele (8 copies of the observed allele are initially placed in the pop. here). Simulation results align with equation.*

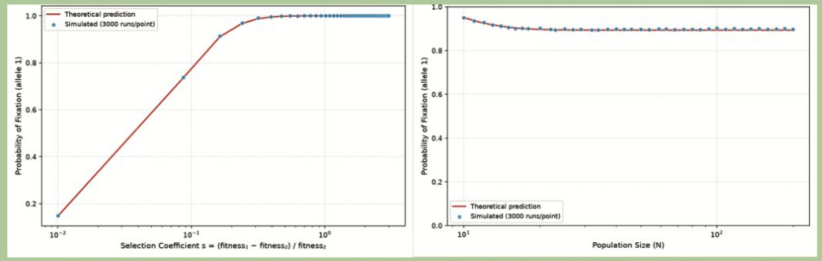


Figure 4: Probability of fixation as a function of selection coefficient (left, pop. size of 300) and population size (right, selection coefficient of 0.15), simulated from two-allele time invariant fitness model compared to Moran model $P = \frac{1 - \left(\frac{1}{r}\right)^i}{1 - \left(\frac{1}{r}\right)^N}$ *which is uniquely an exact equation, not an estimation, and works for all initial conditions. 8 copies of observed allele are initially present. Simulation results align with equation.*

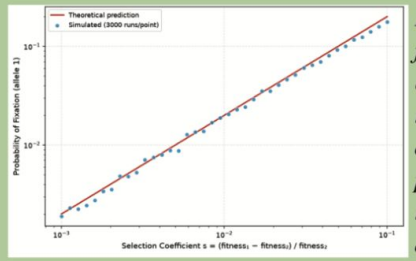
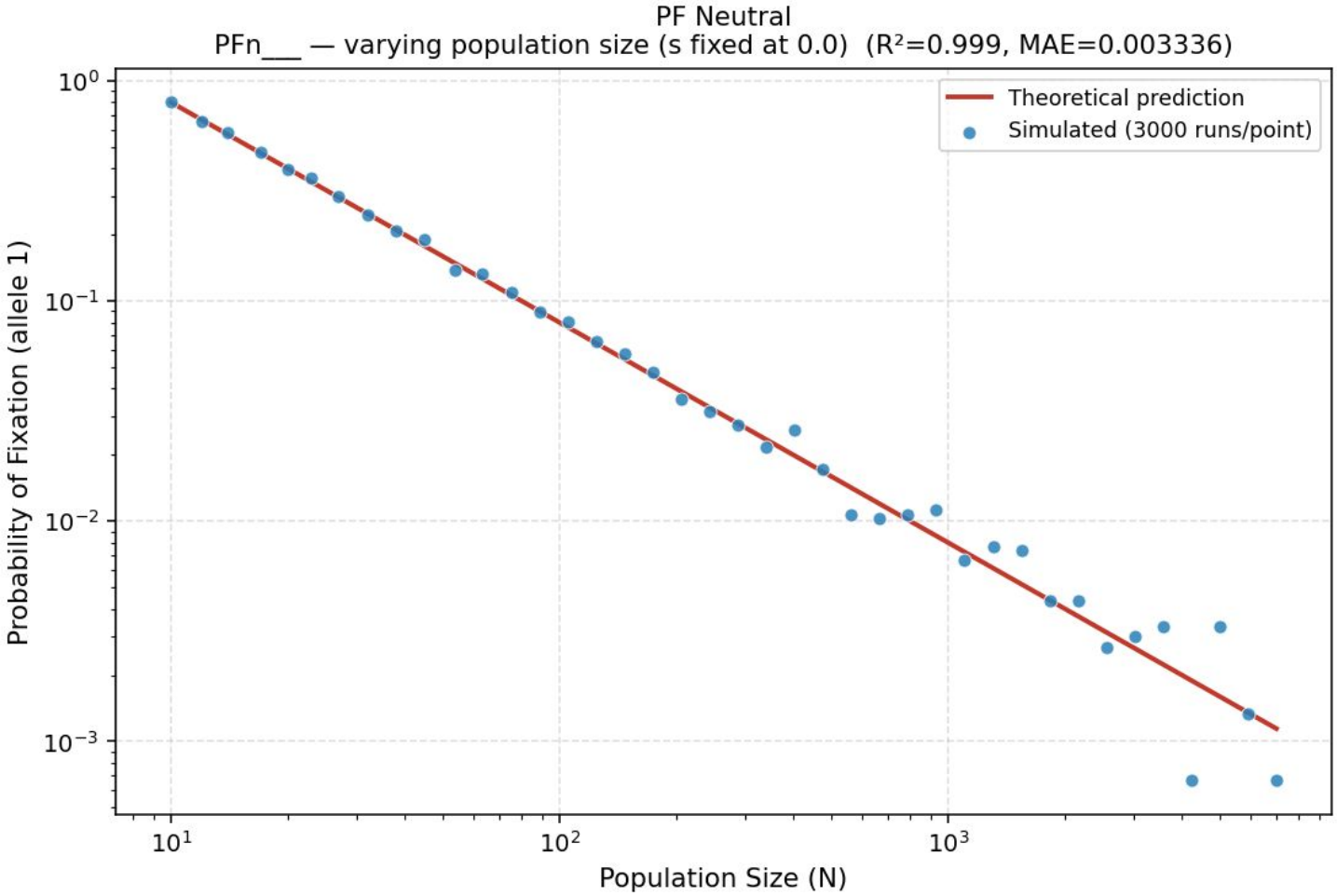
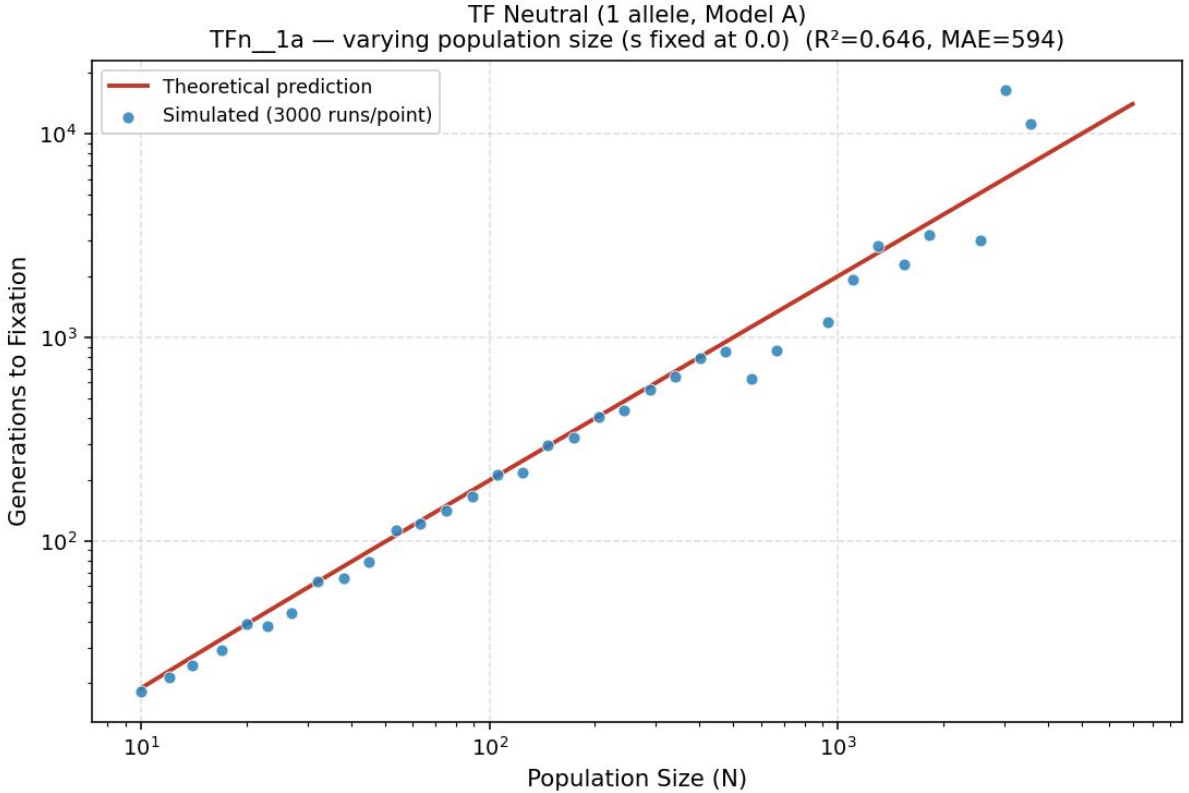


Figure 5: Probability of fixation as a function of selection coefficient (pop. size of 5000), simulated from two-allele, time invariant fitness model compared to expected $P \approx 2s$. *Prediction for large populations with one weakly beneficial initial allele introduced. Simulation results are consistent with known equation.*

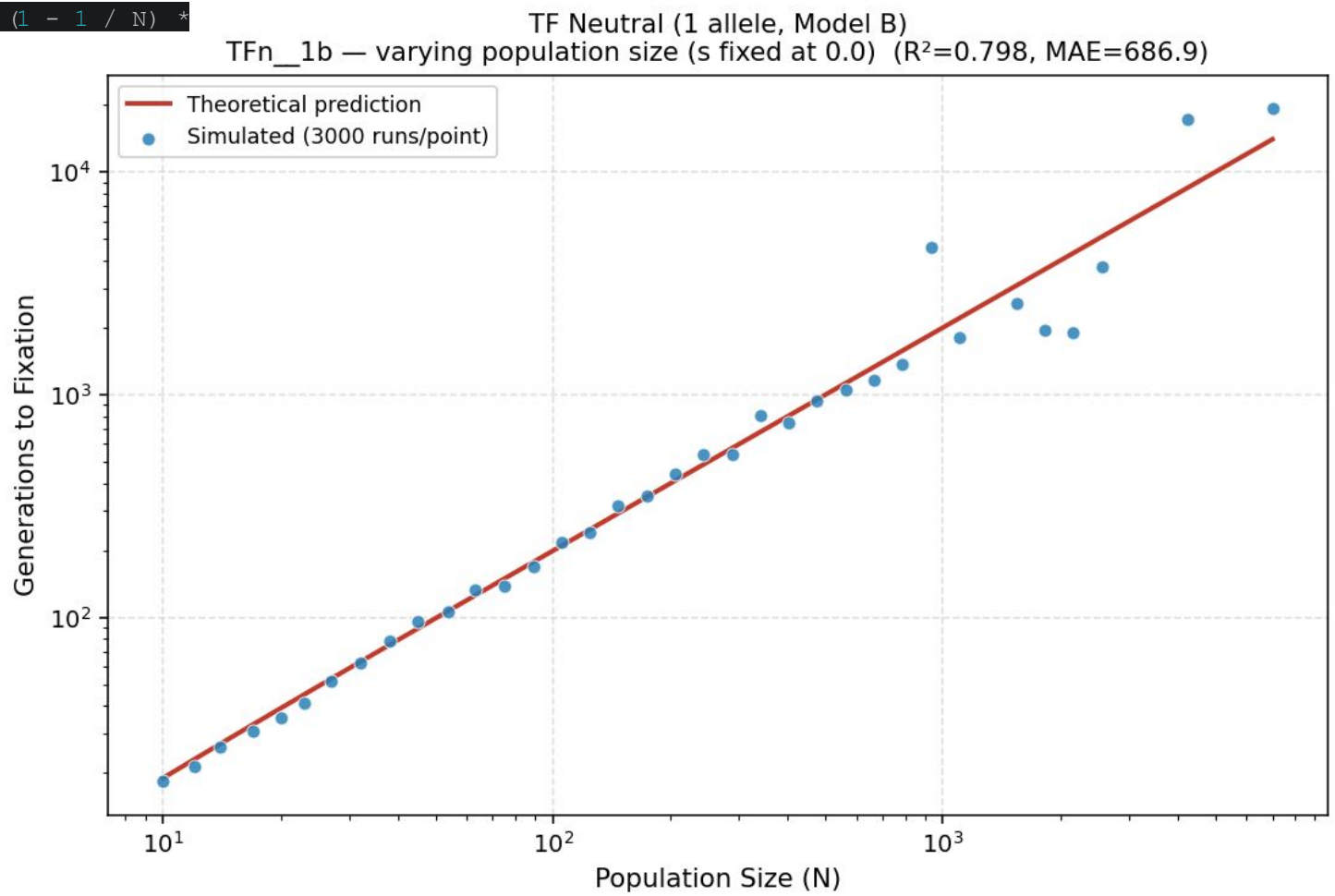
```
expected = size1 / N
```



```
Ne = N
p = size1 / N
if p == 0:
    expected = 2 * Ne
elif p == 1:
    expected = 0.
else:
    expected = -2 * Ne * (1-p) *
np.log1p(-p) / p # equation 14
```



```
Ne = N
expected = -2 * N * Ne * (1 - 1 / N) *
np.log1p(-1 / N)
```



```
r = fitness1 / fitness2
expected = stable_ratio(r, size1,
size2)

def stable_ratio(r, s1, s2):
    if not (float(s1).is_integer() and
float(s2).is_integer()):
        raise ValueError("s1 and s2
must be integers when r can be
negative")
    s1, s2 = int(s1), int(s2)

    if r == 0:
        return 0.
    if r == 1:
        return s1 / (s1 + s2)

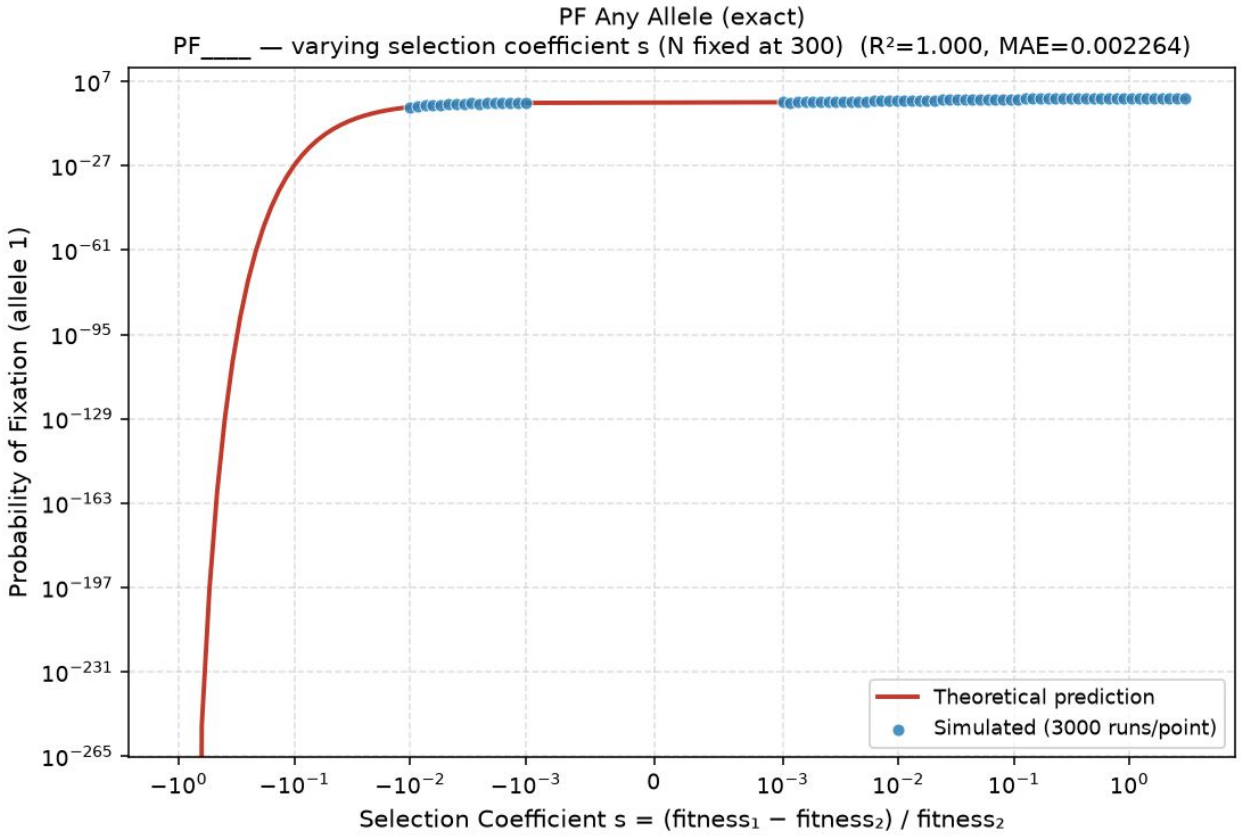
    ln_abs_r = np.log(np.abs(r))

    a = 2 * s1 * ln_abs_r
    b = -2 * s2 * ln_abs_r
    m = max(a, b, 0.0)

    num = np.exp(a - m) - np.exp(-m)
    den = np.exp(a - m) - np.exp(b - m)

    if den == 0.0:
        raise ZeroDivisionError("r**s1
== r**-s2; expression is undefined for
these s1, s2")

    return num / den
```

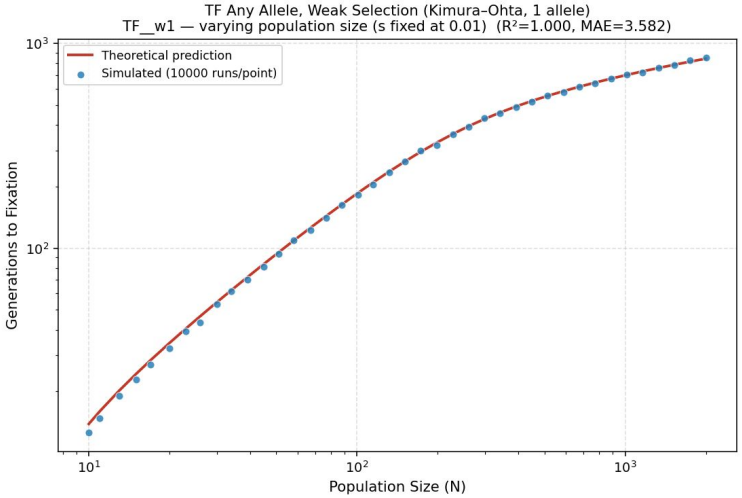


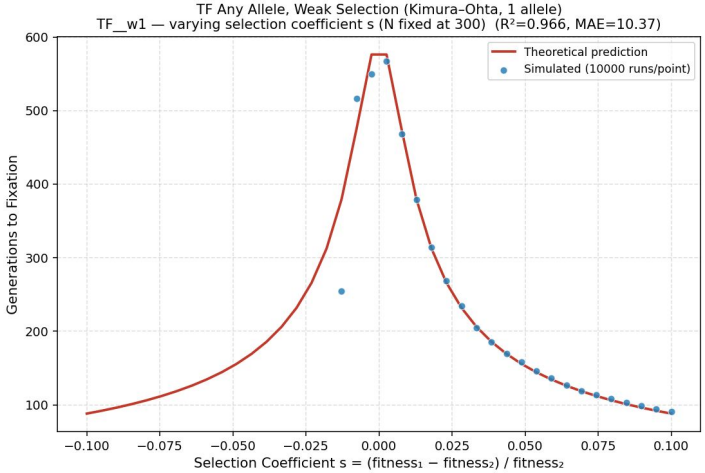
New results:

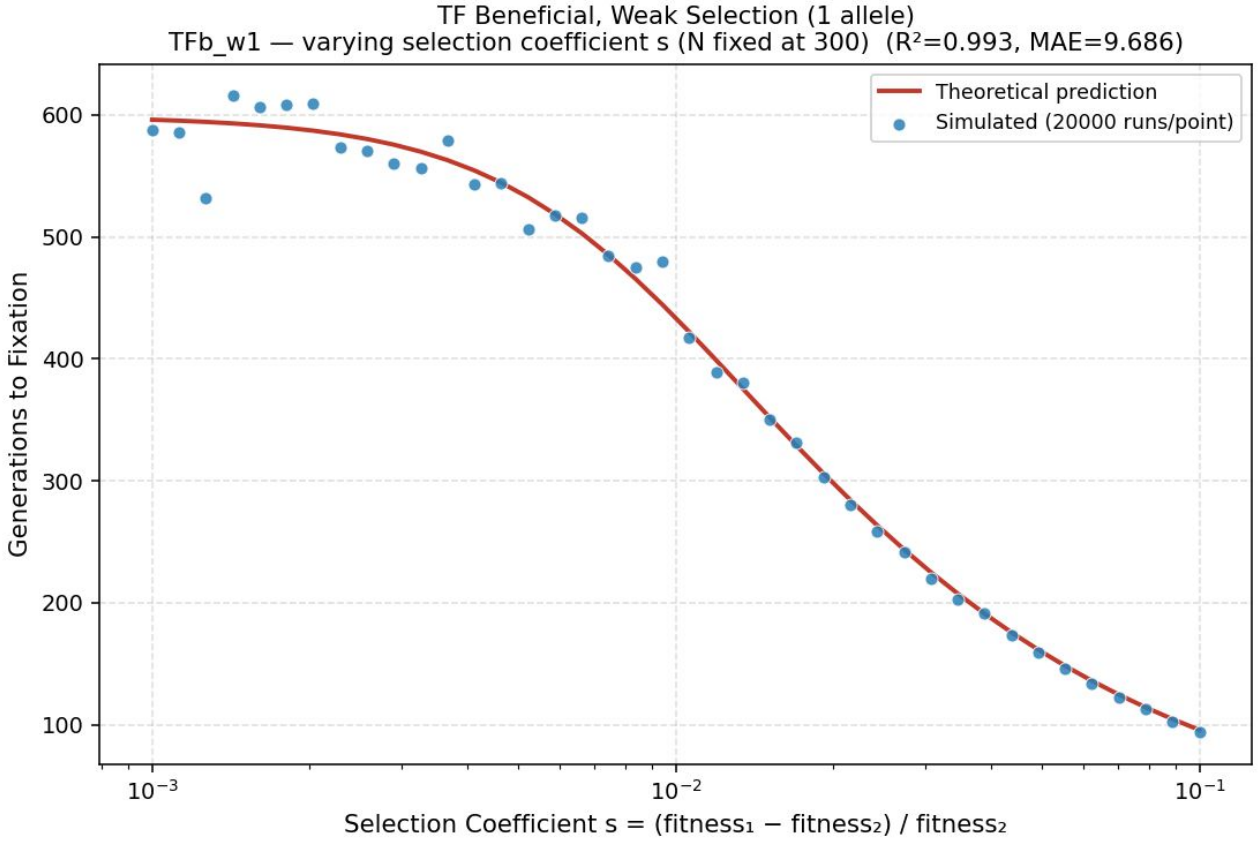
```
N = size1 + size2
s = (fitness1 - fitness2) / fitness2

expected = kimura_ohta_tfix(N, s,
size1)
```

- `TF__w1` calls `tfix_kimura_ohta` with the paper's linear s = (f1−f2)/f2







- `TFb_w1` calls `tfix_kimura_ohta` with s = ln(f1/f2), through `diffusion_s`.

```
expected = tfix_kimura_ohta(N, s,
i0=size1, include_J2=True)
```

Thus from (12), the average number of generations until fixation (excluding the cases of loss) is

$$\bar{t}_1(p) = -\frac{1}{p}\{4N_e(1-p)\log_e(1-p)\}. \qquad (14)$$

```
def tfix_kimura_ohta(N, s, i0=1, include_J2=True):
    """
    Mean time to fixation, conditional on fixation, for a haploid WF population
    (c = 1, Ne = N).

    Kimura & Ohta (1969) eq. 17:  t1(p) = J1 + (1-u(p))/u(p) * J2
    Otto & Whitlock eq. 5 is the J1 term alone, with p = 1/(cNe)
    (set include_J2=False to reproduce eq. 5 exactly).

    Uses the substitution y = logit(x), so dx = x(1-x) dy. This cancels the
    1/(x(1-x)) in both integrands and leaves bounded, overflow-free integrands.
    """
    p = i0 / N
    if s == 0:
        # Kimura & Ohta eq. 14 translated to haploid (4Ne -> 2N)
        return -2 * N * (1 - p) * np.log1p(-p) / p

    # The conditional fixation time is symmetric in s (Otto & Whitlock;
    # Maruyama & Kimura 1974), so deleterious alleles use |s|.
    s = abs(s)
    a = 2 * N * s                  # 2S in Kimura & Ohta, 2cNe*s in Otto & Whitlock
    den = -np.expm1(-a)            # 1 - e^{-2S}

    # J1 integrand: (e^{ax}-1)(e^{a(1-x)}-1) / (e^a - 1), rewritten stably
    def j1(y):
        x, one_minus_x = expit(y), expit(-y)
        return (-np.expm1(-a * x)) * (-np.expm1(-a * one_minus_x)) / den

    # J2 integrand: (e^{ax}-1)(1-e^{-ax}) / (1 - e^{-a})
    def j2(y):
        x = expit(y)
        return np.expm1(a * x) * (-np.expm1(-a * x)) / den

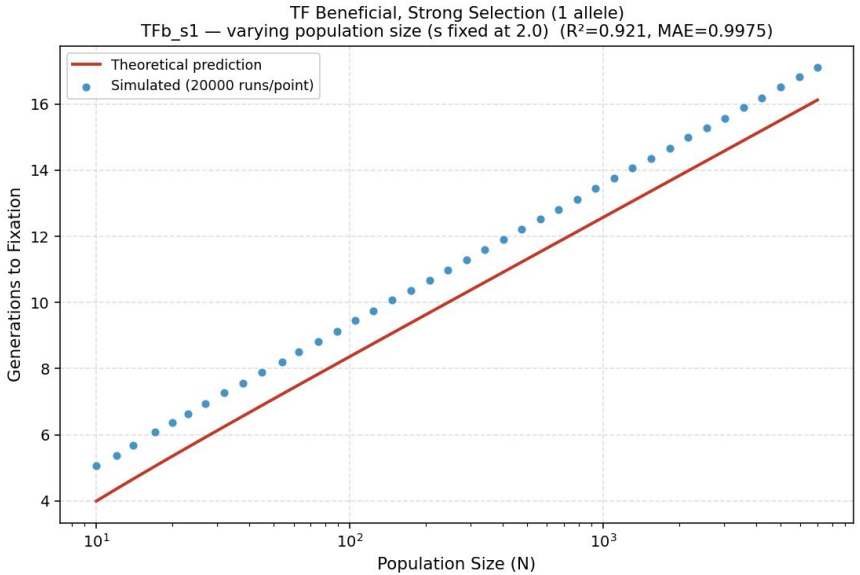
    y_p = np.log(p) - np.log1p(-p)                  # logit(p)
    y_hi = max(y_p + 1.0, np.log(a) + 10.0)         # breakpoint helps quad when a is
huge
    J1 = (quad(j1, y_p, y_hi,  limit=500)[0] + quad(j1, y_hi, np.inf,  limit=500)[0]) / s
    if not include_J2:
        return J1

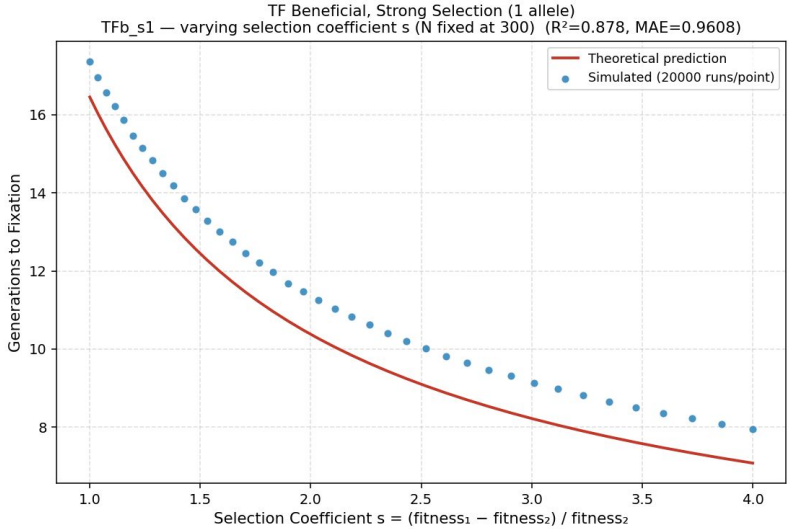
    J2 = quad(j2, -np.inf, y_p,  limit=500)[0] / s
```

```
N = int(INITIAL ALLELE COUNTS.sum())
s = diffusion s(fitness1, fitness2)
# ln(1+s) by default; see section 2

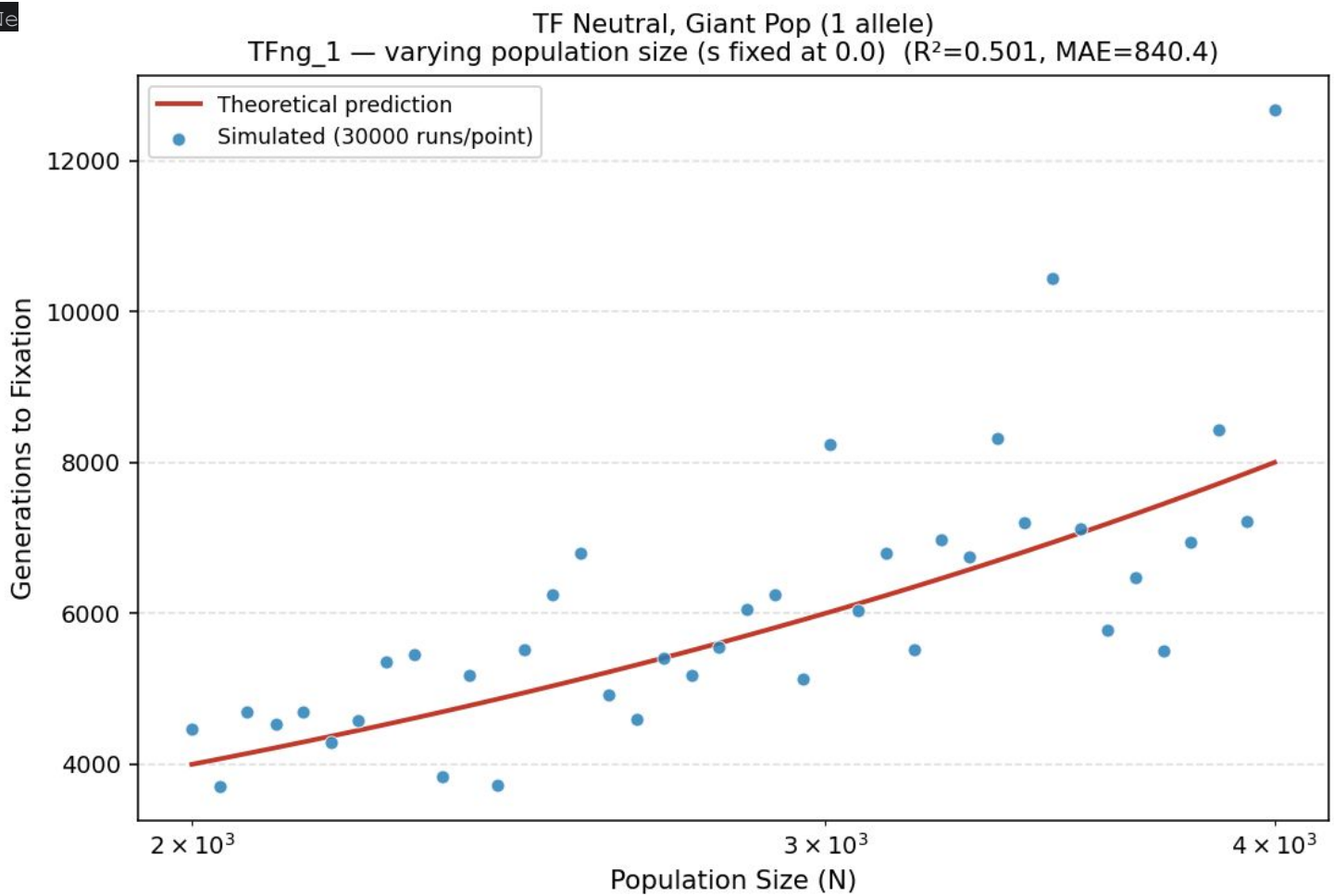
c, Ne = 1, N
# haploid simulator
if s > 0 and (c * Ne - 1) > 0:
    expected = 2 * np.log(c * Ne - 1) /
s
```

the theory "seems to underestimate the actual value slightly… may be due to the existence of the denominator in formula (18) that effectively decreases the value of s."



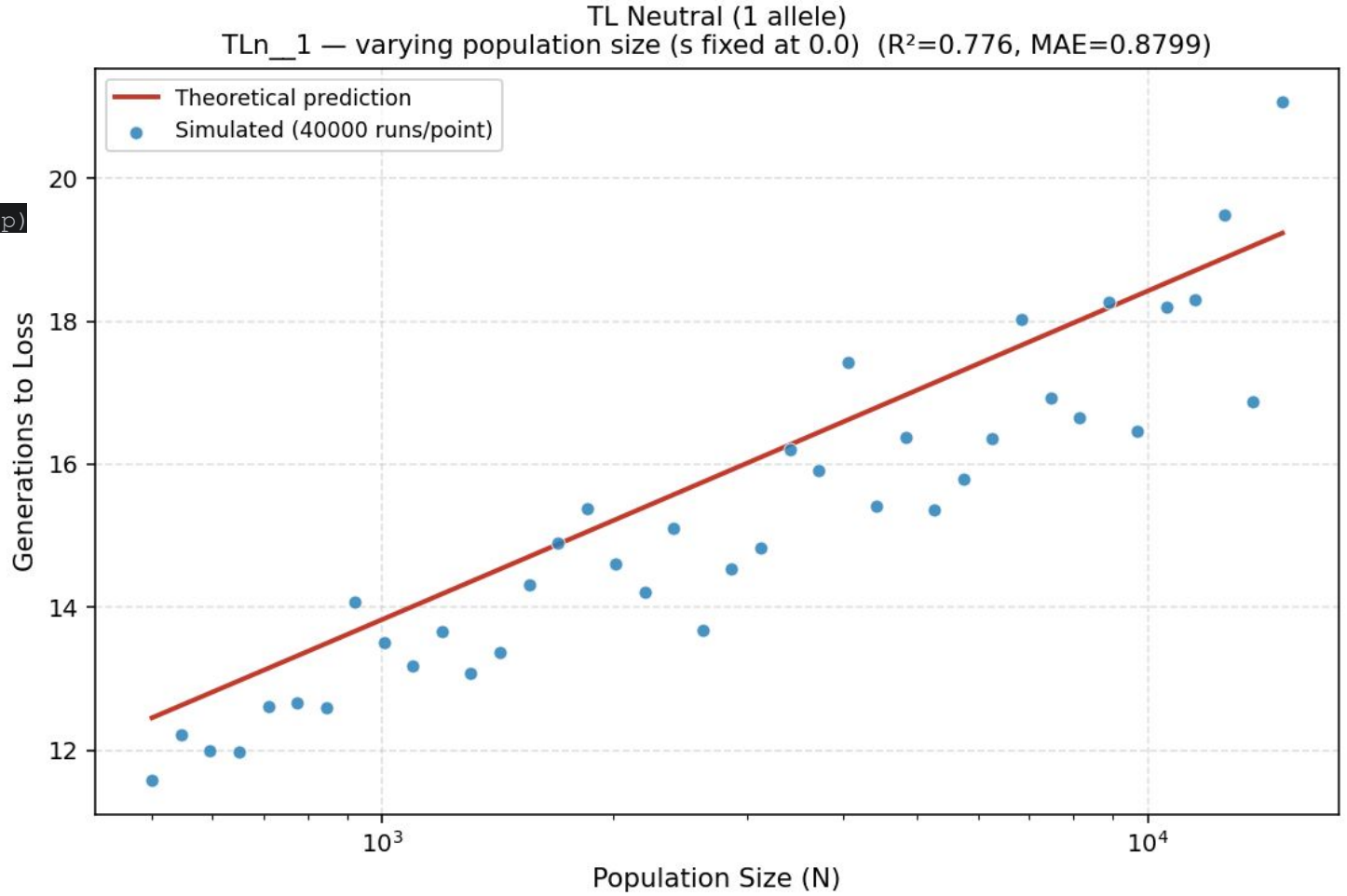


```
Ne = N
expected = 2 * Ne
```



```
Ne = N
p = size1 / N
if p == 0:
    expected = 0.
elif p == 1:
    expected = 2 * Ne
else:
    expected =
-2*Ne*(p/(1-p))*np.log(p)
```

overestimate expected

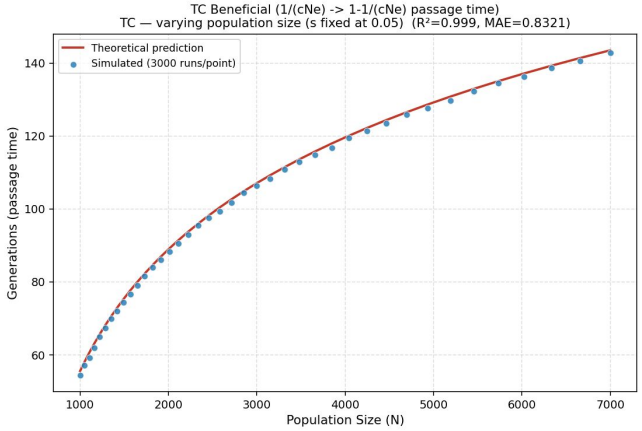


```
function that analyzes time for the
beneficial allele to go from a
concentration 1/(cNe) to 1-1/(cNe) is
called TC


N = int(INITIAL ALLELE COUNTS.sum())
s = diffusion s(fitness1, fitness2)
# ln(f1/f2), matches your WF update
c, Ne = 1, N

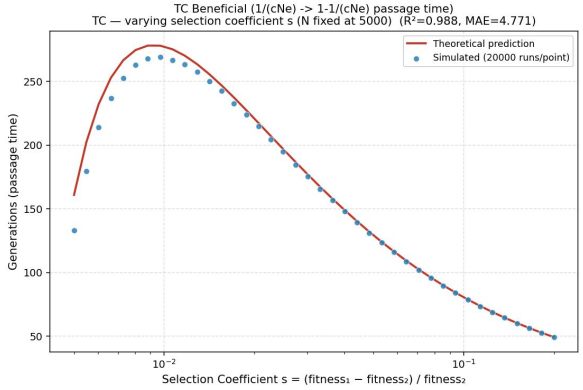
# Literal eq. 6 (window 1/(cNe) ->
1-1/(cNe)), kept for reference only
results['expected TC eq6 literal'] = 2
* np.log(c * Ne - 1) / s if (s > 0 and
c * Ne > 2) else np.nan

# Eq. 6 applied to a drift-safe window
expected, x1, x2 = np.nan, np.nan,
np.nan
if s > 0:
    x1 = TC K / (c * Ne * s)
    x2 = 1 - x1
    if x1 < 0.5 and size1 / N < x1:
# window must exist and start above
the initial freq
        expected = (np.log(x2 / (1 -
x2)) - np.log(x1 / (1 - x1))) / s
```



$$\bar{t} \approx \frac{2\ln(cN_e - 1)}{s} \qquad (6)$$

Equation (6) also equals the time for a beneficial allele to rise from frequency $1/(cN_e)$ to $1-1/(cN_e)$ in a model that ignores drift (see Crow and Kimura, 1970, eqn (5.3.13)).



```python
def fft_terms(history: np.ndarray, Ff: List[Callable[[int], float]], N:
int):
    """
    Per-generation quantities for Fisher's fundamental theorem, for one run.

    For each transition t -> t+1 (fitness evaluated at time t, exactly as in
    wrightfishersampling), returns:
      delta      : observed change in mean fitness, W̄(p[t+1]) - W̄(p[t]),
                   both measured with the generation-t fitness values
                   (Fisher's 'change due to natural selection')
      predicted  : Var_t(w) / W̄_t            (exact FFT, discrete
generations)
      naive      : Var_t(w)                  (the form used in the old FFT)
      drift_var  : Var[ΔW̄ | p_t] = Var_{p'}(w) / N   (multinomial sampling
noise)
    Works for any number of alleles.
    """
    T = history.shape[0] - 1
    if T == 0:
        empty = np.empty(0)
        return empty, empty, empty, empty

    p = history[:-1] / N                      # (T, K) frequencies
before selection
    p_next = history[1:] / N                  # (T, K) frequencies
after sampling
    W = np.array([[f(t) for f in Ff] for t in range(T)], dtype=float)
# (T, K)

    wbar = np.sum(p * W, axis=1)
    var_w = np.sum(p * W**2, axis=1) - wbar**2
    var_w = np.maximum(var_w, 0.0)            # guard tiny negative
round-off

    delta = np.sum(p_next * W, axis=1) - wbar
    predicted = var_w / wbar

    p_sel = p * W / wbar[:, None]             # expected frequencies
after selection
    m_sel = np.sum(p_sel * W, axis=1)         # equals wbar + predicted
    drift_var = np.maximum(np.sum(p_sel * W**2, axis=1) - m_sel**2, 0.0)
/ N

    return delta, predicted, var_w, drift_var


def FFT(INITIAL_ALLELE_COUNTS, FITNESS_FUNCTIONS, NUM_RUNS,
use_parallel_core: bool=False,
        executor: ProcessPoolExecutor=None, alpha: float=0.01) -> Dict:
    """
    Tests whether the simulation obeys Fisher's fundamental theorem:
        E[ΔW̄ | p_t] = Var_t(w) / W̄_t

    Test 1 (integrated, independent runs): per run, R = Σ_t (ΔW̄_t -
Var_t/W̄_t).
        R is a stopped martingale, so E[R] = 0 exactly. One-sample t-test
across runs.
    Test 2 (per generation): z_t = (ΔW̄_t - Var_t/W̄_t) / sqrt(Var[ΔW̄_
p_t]).
        E[z] = 0 tests selection; E[z²] = 1 tests that drift is
binomial/multinomial.
    """
    results = {}
    N = int(np.sum(INITIAL_ALLELE_COUNTS))

    sim_data = run_multiple_simulations(INITIAL_ALLELE_COUNTS,
FITNESS_FUNCTIONS, NUM_RUNS,

use_parallel_core=use_parallel_core, executor=executor)

    run_sums, deltas, preds, naives, zs = [], [], [], [], []
    for run in sim_data:
        d, pr, nv, dv = fft_terms(run['allele_counts_history'],
FITNESS_FUNCTIONS, N)
        if d.size == 0:
            continue
        run_sums.append(np.sum(d - pr))
        deltas.append(d); preds.append(pr); naives.append(nv)
        mask = dv > 1e-15                     # zero only if all
fitnesses are equal
        zs.append((d[mask] - pr[mask]) / np.sqrt(dv[mask]))

    if not deltas:
        results.update(difference=np.nan, passed=False)
        return results

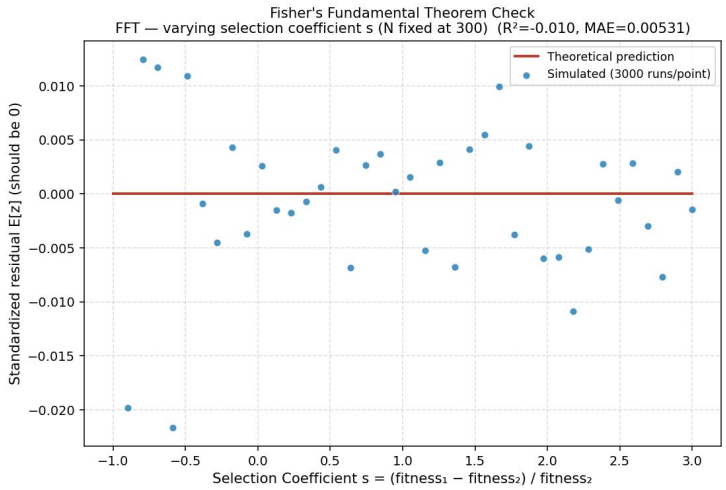
    delta = np.concatenate(deltas)
    pred = np.concatenate(preds)
    naive = np.concatenate(naives)
    z = np.concatenate(zs) if zs else np.empty(0)
    run_sums = np.asarray(run_sums)

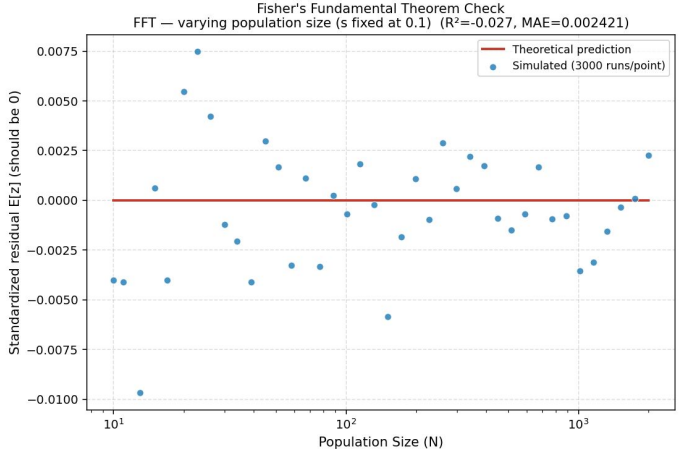
    # Pooled summaries (kept for compatibility with your
sweep/plot code)
    results['difference'] = float(np.mean(delta - pred))
# should be ≈ 0
    results['difference_naive'] = float(np.mean(delta - naive))
# old test: ΔW̄ - Var
    results['mean_delta_wbar'] = float(np.mean(delta))
    results['mean_predicted'] = float(np.mean(pred))
    results['n_generations'] = int(delta.size)

    # Test 1: integrated residual per run (independent samples)
    results['run_sum_mean'] = float(np.mean(run_sums))
    if run_sums.size > 1 and np.std(run_sums) > 0:
        results['run_sum_pvalue'] = float(ttest_1samp(run_sums,
0.0).pvalue)
    else:
        results['run_sum_pvalue'] = 1.0       # identical runs
(e.g. neutral: all residuals 0)

    # Test 2: standardized per-generation residuals
    if z.size > 1:
        results['z_mean'] = float(np.mean(z))
        results['z_mean_pvalue'] = float(2 *
norm.sf(abs(np.mean(z)) * np.sqrt(z.size)))
        z2 = z**2
        results['z_var'] = float(np.mean(z2))
# should be ≈ 1
        z2_se = np.std(z2, ddof=1) / np.sqrt(z.size)
        results['z_var_pvalue'] = float(2 *
norm.sf(abs(np.mean(z2) - 1) / z2_se)) if z2_se > 0 else 1.0
    else:
        results.update(z_mean=np.nan, z_mean_pvalue=1.0,
z_var=np.nan, z_var_pvalue=1.0)

    results['passed'] = bool(results['run_sum_pvalue'] > alpha
and
                             results['z_mean_pvalue'] > alpha and
                             results['z_var_pvalue'] > alpha)
    return results
```

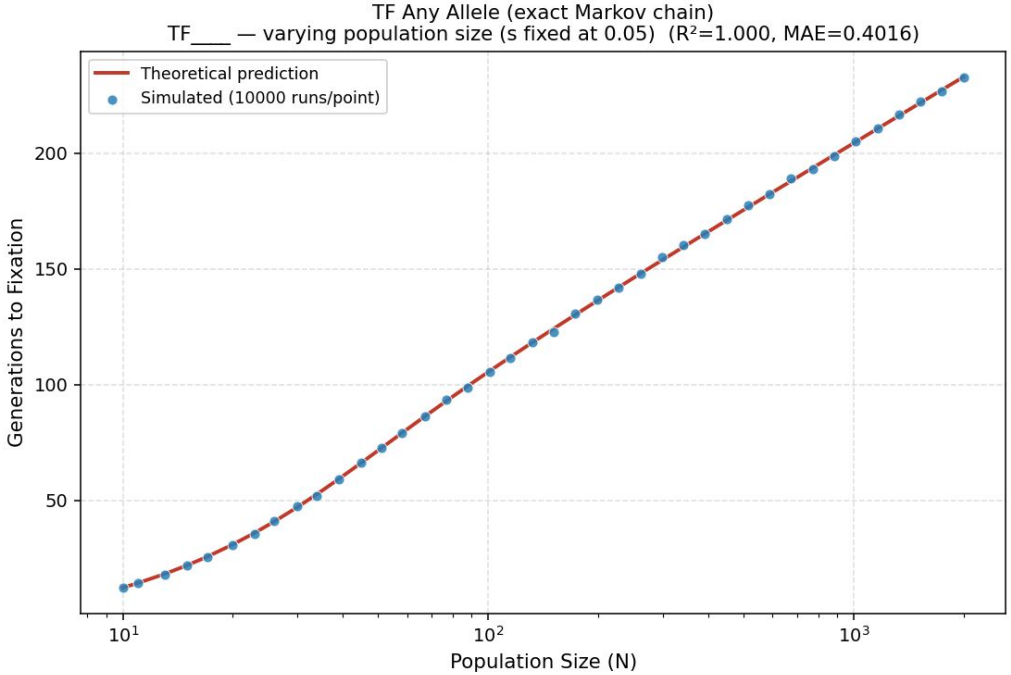


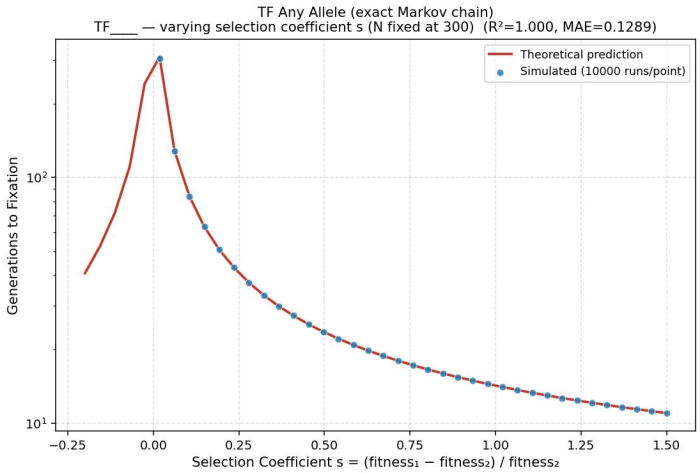


```
N = size1 + size2
s = (fitness1 - fitness2) / fitness2

if N <= EXACT_CHAIN_MAX_N:
    expected =
exact_wf_conditional_tfix(N, s, size1)
```

This is not a biological known equations check, this is purely a code check






```
def exact_wf_conditional_tfix(N, s, i0=1):
    """Exact mean time to fixation (conditioned on
fixation) of the discrete WF chain."""
    i = np.arange(N + 1)
    x = i * (1 + s) / (i * (1 + s) + (N - i))
# same as the sim's pvals
    P = binom.pmf(i[None, :], N, x[:, None])
# P[i, j]
    Q, R = P[1:N, 1:N], P[1:N, N]
    I = np.eye(N - 1)
    u = np.linalg.solve(I - Q, R)
# fixation probabilities
    Qc = Q * u[None, :] / u[:, None]
# chain conditioned on fixation
    T = np.linalg.solve(I - Qc, np.ones(N -1))
# expected steps to absorption
    return T[i0 - 1]
```